# Registro de Decisiones de Arquitectura (ADR)

Plataforma de Farmacología Gamificada — Lote 4 (ADR-0011 a ADR-0015)

*Versión 1 — OFICIAL*

**Fecha del documento:** 27 de octubre de 2026

Decisiones tomadas al implementar la base compartida del monolito y el primer módulo (Autenticación). Complementan los lotes 1 a 3 sin reemplazar ninguna decisión anterior.

## ADR-0011: Framework HTTP: Express 5

**Fecha:** 27 de octubre de 2026

**Estado:** Aceptada

### Contexto

El backend en Node.js con TypeScript (ADR-0001) necesita un framework HTTP para exponer la API. La estructura de carpetas ya contemplaba una capa de rutas delgada que solo conecta HTTP con la fachada de cada módulo (ADR-0004), por lo que el framework no debe imponer su propia arquitectura sobre la del monolito modular.

### Opciones consideradas

- Express 5: minimalista, el más usado del ecosistema Node.js.
- Fastify: más rápido y con validación por esquemas integrada.
- NestJS: framework completo y prescriptivo (módulos, decoradores, inyección de dependencias propia).

### Decisión

Se adopta Express 5 como framework HTTP. Las rutas (src/rutas/) solo traducen la petición HTTP a una llamada a la fachada del módulo y devuelven su resultado; no contienen lógica de negocio ni validación.

### Trade-offs

| Aspecto | Análisis |
|---|---|
| A favor | Es el framework con más documentación, ejemplos y material de aprendizaje, lo que reduce la curva para un equipo de 1-2 personas sin experiencia previa en Node.js; la versión 5 propaga automáticamente los errores de funciones async al manejador de errores (antes había que envolverlas); no impone una arquitectura, así que respeta las capas y fachadas definidas en ADR-0004. |

| Aspecto | Análisis |
| --- | --- |
| En contra / riesgos | Es más lento que Fastify en pruebas de rendimiento puro (aunque no es el cuello de botella esperado frente a MySQL); no trae validación integrada, por lo que se complementa con Zod (ADR-0009). |

## Alternativa descartada

NestJS habría aportado mucha estructura de fábrica, pero duplica conceptos que ya definimos (módulos, inyección de dependencias) y añade una curva de aprendizaje alta. Fastify es una opción válida, pero con menos material de aprendizaje; puede reconsiderarse si el rendimiento HTTP llega a ser un problema.

# ADR-0012: Formato único de errores y validación dentro del módulo

**Fecha:** 27 de octubre de 2026

**Estado:** Aceptada

## Contexto

Con varios módulos respondiendo a la misma API, se necesita que todos los errores lleguen al cliente con el mismo formato, sin exponer detalles internos (trazas, mensajes de la base de datos). También hay que decidir dónde se ejecuta la validación con Zod (ADR-0009): en las rutas o dentro de cada módulo.

## Opciones consideradas

- Cada ruta captura sus errores y arma su propia respuesta.
- Errores tipados de aplicación + un único manejador de errores; validación en un middleware de rutas.
- Errores tipados de aplicación + un único manejador de errores; validación dentro de cada caso de uso del módulo.

## Decisión

Se define una jerarquía de errores (ErrorAplicacion y sus subclases ErrorValidacion 400, ErrorNoAutenticado 401, ErrorNoAutorizado 403, ErrorNoEncontrado 404, ErrorConflicto 409), cada uno con su código HTTP y un código interno estable. Un único manejador de errores responde siempre con el formato { error: { codigo, mensaje, detalles? } }; cualquier error no controlado se registra en el servidor y se responde como 500 genérico.

La validación con Zod ocurre dentro de cada caso de uso del módulo (vía validarConEsquema), no en las rutas: así la fachada queda protegida sin importar quién la llame (una ruta HTTP u otro módulo).

## Trade-offs

| Aspecto | Análisis |
|---|---|
| A favor | El frontend puede tratar todos los errores de la misma manera; nunca se filtran detalles internos; un módulo no puede recibir datos sin validar aunque se le llame desde otro módulo; las pruebas pueden verificar errores por su tipo. |
| En contra / riesgos | Hay que acordarse de lanzar errores de la jerarquía en lugar de errores genéricos; la validación en el caso de uso hace que la ruta no conozca la forma exacta del cuerpo esperado (vive en el esquema Zod del módulo). |

## Alternativa descartada

Que cada ruta arme su respuesta de error es más rápido al inicio, pero produce formatos inconsistentes y riesgo de filtrar información interna. Validar solo en un middleware de rutas dejaría la fachada sin protección cuando otro módulo la llame directamente.

# ADR-0013: Estrategia de pruebas: node:test + tsx con dependencias en memoria

**Fecha:** 27 de octubre de 2026

**Estado:** Aceptada

## Contexto

Se necesita probar la lógica de cada módulo de forma rápida y sin depender de una base de datos MySQL levantada. La inyección de dependencias definida en ADR-0004 (regla 5) permite reemplazar la infraestructura por implementaciones falsas.

## Opciones consideradas

- Jest: el framework de pruebas más popular, requiere configuración extra para TypeScript.
- Vitest: moderno y rápido, añade varias dependencias.
- node:test (incluido en Node.js) ejecutado con tsx para soportar TypeScript sin compilar.

## Decisión

Se adopta el ejecutor de pruebas nativo de Node.js (node:test y node:assert) con tsx. Las pruebas de casos de uso usan repositorios en memoria en lugar de Prisma, pero mantienen bcrypt y JWT reales. Las pruebas viven en tests/<modulo>/ y se ejecutan con npm test.

## Trade-offs

| Aspecto | Análisis |
|---|---|
| A favor | Cero configuración y casi ninguna dependencia extra; las pruebas corren en segundos y sin base de datos; obliga a respetar los puertos (interfaces) de cada módulo, porque es la única forma de inyectar las versiones en memoria. |
| En contra / riesgos | Tiene menos utilidades que Jest o Vitest (por ejemplo, simulación de módulos o cobertura con reporte visual); las pruebas en memoria no detectan errores propios de MySQL, por lo que más adelante se necesitarán pruebas de integración con una base de datos real. |

## Alternativa descartada

Jest y Vitest son más completos, pero añaden configuración y dependencias que hoy no se justifican. Si las necesidades de pruebas crecen (cobertura, simulaciones complejas), Vitest es la primera alternativa a evaluar.

# ADR-0014: Prisma 7 con adaptador MariaDB y versiones estables

**Fecha:** 27 de octubre de 2026

**Estado:** Aceptada

## Contexto

Prisma 7 (ADR-0007) cambió su forma de conectarse: la URL de la base de datos sale del esquema y pasa a prisma.config.ts, y en tiempo de ejecución el cliente necesita un adaptador de conexión. Además, al instalar se detectó que la etiqueta "latest" de Prisma en npm apuntaba a una versión candidata (8.0.0-rc.28), no a una versión estable.

## Opciones consideradas

- Aceptar la versión que npm marque como "latest", sea cual sea.
- Fijar Prisma (CLI, cliente y adaptador) en la misma versión estable y actualizar de forma deliberada.

## Decisión

Se fijan prisma, @prisma/client y @prisma/adapter-mariadb en la misma versión estable (7.10.1). La conexión usa el adaptador MariaDB (compatible con MySQL) con el pool configurado desde las variables de entorno (ADR-0003). Se usa un único cliente Prisma para todo el monolito, inyectado en el repositorio de cada módulo.

El cliente generado (src/generado/prisma) no se sube a Git: se regenera con npm run prisma:generar. Las tablas llevan el prefijo de su módulo y no hay llaves foráneas entre módulos (ADR-0004).

## Trade-offs

| Aspecto | Análisis |
|---|---|
| A favor | Evita errores por mezclar versiones o usar una versión candidata en producción; el pool de conexiones queda controlado por la configuración validada al arrancar; el repositorio Git no se llena de código generado. |
| En contra / riesgos | Hay que actualizar las tres dependencias de Prisma juntas y revisar sus notas de versión; quien clone el repositorio debe ejecutar prisma generate antes de compilar. |

## Alternativa descartada

Aceptar la etiqueta "latest" sin revisar es más cómodo, pero hoy habría instalado una versión candidata incompatible con el cliente, con riesgo de errores difíciles de diagnosticar.

# ADR-0015: Convenciones de código y de nombres

**Fecha:** 27 de octubre de 2026

**Estado:** Aceptada

## Contexto

La estructura de carpetas (borrador v1) dejó pendiente la convención de nombres de archivos, y el lenguaje ubicuo (ADR-0005) exige nombres del dominio en español. Con el primer módulo ya escrito, conviene dejar las convenciones por escrito para mantener la consistencia.

## Opciones consideradas

- Nombres de archivos en kebab-case (ej. casos-de-uso.ts).
- Nombres de archivos en camelCase (ej. casosDeUso.ts), igual que funciones y variables.

## Decisión

Se adoptan estas convenciones: archivos y carpetas en camelCase; clases en PascalCase; funciones y variables en camelCase; tablas y columnas de la base de datos en snake_case con prefijo del módulo (ej. autenticacion_usuarios); todos los nombres en español y completos, sin abreviaturas, siguiendo el glosario.

TypeScript se compila en modo estricto (strict, noUncheckedIndexedAccess, noImplicitOverride). Cada módulo sigue la forma dominio/ (modelo y puertos), aplicacion/ (casos de uso y esquemas Zod), infraestructura/ (Prisma, bcrypt, JWT, etc.) y fachada.ts.

Los commits siguen el formato de Conventional Commits en español (feat, fix, chore, docs, con el módulo entre paréntesis cuando aplica).

## Trade-offs

| Aspecto | Análisis |
|---|---|
| A favor | Un solo estilo de nombres en todo el código TypeScript; el modo estricto detecta muchos errores al compilar; el historial de Git se lee fácilmente por tipo de cambio. |
| En contra / riesgos | camelCase en archivos puede dar problemas en sistemas de archivos que no distinguen mayúsculas y minúsculas si se renombra un archivo solo cambiando mayúsculas (mitigado con forceConsistentCasingInFileNames). |

## Alternativa descartada

kebab-case es muy común en proyectos Node.js, pero obligaría a mantener dos estilos de nombres (archivos y código) en un proyecto que ya definió camelCase como convención en los requisitos.